Express.js — Day 9

Multiple Middleware, Execution Order & Modifying the Request

■ Day 9 Goal

Understand how Express handles multiple middleware functions, why middleware order matters, how **next()** moves the request forward, and how middleware can add information to the **req** object for later middleware and route handlers to use.

1. Multiple Middleware Functions

An Express application can have more than one middleware function. Each middleware gets a chance to perform its task and then calls **next()** to pass control to the next middleware or route handler.

```
app.use((req, res, next) => {
    console.log("Middleware 1");
    next();
});

app.use((req, res, next) => {
    console.log("Middleware 2");
    next();
});

app.get("/", (req, res) => {
    res.send("Hello Express");
});
```

Request flow

GET / → **Middleware 1** → **next()** → **Middleware 2** → **next()** → **Route** → **Response**

2. Middleware Execution Order ■

Express processes middleware according to the order in which it is registered in the application.

Registered order	Execution order
Middleware 1	1st
Middleware 2	2nd
Route handler	3rd

If you swap the two middleware registrations, their execution order also swaps. This is why middleware order is important in real Express applications.

3. Why next() Matters

Calling **next()** tells Express that the current middleware has finished its work and that the request should continue through the middleware chain.

```
app.use((req, res, next) => {
    console.log("Middleware 1");
```

```
    next();
});
```

If middleware neither calls **next()** nor sends a response, the request can remain hanging. A middleware must either finish the request or pass control forward.

4. Middleware Can Modify the Request ■

Middleware can add information to the incoming request object. Because the request continues through the middleware chain, later middleware and route handlers can access that added information.

```
app.use((req, res, next) => {
    req.message = "Hello from middleware";
    next();
});

app.get("/", (req, res) => {
    res.send(req.message);
});
```

What happens here?

1	A request arrives at GET /.
2	Middleware receives the request.
3	Middleware adds req.message.
4	next() passes the same request forward.
5	The route reads req.message.
6	The route sends the value to the client.

5. Day 9 Practical Test

In the Day 9 exercise, two middleware functions were registered before the route:

```
app.use((req, res, next) => {
    console.log("Middleware 1");
    next();
});

app.use((req, res, next) => {
    console.log("Middleware 2");
    next();
});

app.get("/", (req, res) => {
    res.send("Hello Express");
});
```

When GET / was sent from Postman, the terminal showed Middleware 1 followed by Middleware 2, and Postman received **Hello Express**.

6. Day 9 Request Modification Test

```
app.use((req, res, next) => {
    req.message = "Hello from middleware";
    next();
```

```
});

app.get("/", (req, res) => {
    res.send(req.message);
});
```

Postman returned **Hello from middleware**. This demonstrated that middleware can add data to the request and a later route handler can read it.

7. Important Mental Model

Middleware is part of a request-processing chain. It can inspect the request, modify the request, perform common work, and then use **next()** to continue.

```
Incoming Request
       ↓
Middleware 1
       ↓ next()
Middleware 2
       ↓ next()
Route Handler
       ↓
Response
```

8. Day 9 Knowledge Check — Answers

1. What is the execution order?

Middleware 1 → Middleware 2 → Route handler, because Express follows registration order.

2. What if middleware does not call next() and does not send a response?

The request can hang because Express has not been told to continue and no response has been sent.

3. Can a route access req.user added by middleware?

Yes. Middleware can add information to the request object, and later middleware/routes can access it as the same request continues through the chain.

9. Long-Term Memory Notes

• Middleware can be chained.

• Middleware executes in registration order.

• next() passes control to the next middleware/handler.

• A middleware that neither responds nor calls next() can leave the request hanging.

• Middleware can add properties such as req.user or req.message.

• Later routes can read those properties from the request.

■ Day 9 Complete

Express.js progress: 9 / 15 days → 60%

Day 9 is complete. You have now practiced multiple middleware, middleware ordering, next(), and passing

information forward through the request object.